

Quiz 4

315832025 連怡卉

AI Tools: Codex

請幫我完成底下的要求，圖片是q4-1, q4-2
Quiz 4 影像拼接...

已完成 Quiz4，使用 q4-1.jpg 與 q4-2.jpg。

完成內容：

- 基礎：
 - SIFT 特徵偵測
 - BFMatcher 特徵匹配
 - Low ratio test
 - RANSAC Homography
 - 影像透視變換與重疊區域融合
 - 輸出 `Data/output/q4-stitched.jpg`
- 基礎實際結果：
 - 有效匹配：139
 - RANSAC 內點：128
 - 內點比例：92.09%
 - 判定為成功拼接
- 進階測試：
 - 亮度增加與降低
 - 旋轉 8 度、20 度
 - 減少重疊範圍
 - CLAHE 前處理
 - 顯示匹配數、內點數與成功/失敗結果

其中「亮度降低 60」時只剩 5 個匹配、內點數為 0，因此判定拼接失敗。相關分析也已補充至 [README.md](#)，程式則已寫入 [main.ipynb](#)。

請整理並解釋下Q4的原理與程式撰寫的部分，特別著重在各技術的實現，像是SIFT, RANSAC等等的(直接告訴我就好)

2:08 PM

Q4 影像拼接原理

Q4 使用 q4-1.jpg 和 q4-2.jpg，兩張照片有部分重疊區域。程式的目標是找出兩張影像中相同的位置，再將第二張影像變形到第一張影像的座標系統，最後合併成一張全景圖。

整體流程是：

```
graph TD; A[讀取影像] --> B[SIFT 特徵偵測]; B --> C[特徵描述子匹配]; C --> D[Low ratio test 篩選]; D --> E[RANSAC 估計 Homography]; E --> F[Perspective Transform]; F --> G[影像融合];
```

1. SIFT 特徵偵測

SIFT 的全名是 Scale-Invariant Feature Transform，主要用來找出影像中具有辨識度的特徵，例如：

- 掛畫的角落
- 物體邊緣
- 紋理交界
- 明暗變化明顯的位置

這些特徵點在兩張照片中如果是同一個物體，通常可以被互相對應。

程式實作

```
sift = cv2.SIFT_create(  
    nfeatures=4000,  
    contrastThreshold=0.02  
)  
  
keypoints1, descriptors1 = sift.detectAndCompute(gray1, None)  
keypoints2, descriptors2 = sift.detectAndCompute(gray2, None)
```

SIFT 通常分成兩個部分：

特徵點 Keypoints

`keypoints1` 和 `keypoints2` 儲存影像中的特徵位置，例如：

(x, y) 座標
特徵點的尺度
特徵點的方向

SIFT 會在不同尺度下尋找影像中的穩定區域，因此對影像縮放和旋轉具有一定穩定性。

特徵描述子 Descriptors

`descriptors1` 和 `descriptors2` 是每個特徵點的數值描述，通常是一組 128 維向量。

它不是單純記錄某個像素的顏色，而是記錄特徵點附近的：

- 邊緣方向
- 梯度強度
- 局部紋理
- 鄰近區域的變化

因此，即使影像亮度稍微不同，只要附近的形狀和紋理仍然存在，SIFT 仍有機會找到相同特徵。

2. BFMatcher 特徵匹配

取得兩張影像的 SIFT 描述子後，需要找出哪些特徵彼此相似。

程式使用 BFMatcher：

```
matcher = cv2.BFMatcher(cv2.NORM_L2)
knn_matches = matcher.knnMatch(
    descriptors2,
    descriptors1,
    k=2
)
```

BFMatcher 是 Brute-Force Matcher，做法很直接：

1. 取出第二張影像的一個描述子。
2. 和第一張影像所有描述子計算距離。
3. 找出距離最近的特徵。
4. 同時保留第一近與第二近的結果。

`cv2.NORM_L2` 使用歐氏距離衡量兩個 SIFT 描述子的差異：

$$d(a, b) = \sqrt{\sum_i (a_i - b_i)^2}$$

距離越小，代表兩個局部特徵越相似。

3. Lowe Ratio Test

單純選擇距離最近的特徵並不可靠，因為有些特徵可能和很多地方都很像。

例如：

最近的匹配距離 = 100
第二近的匹配距離 = 105

雖然第一個是最近的，但其實它和第二個差不多，代表這個匹配不夠明確。

程式使用 Lowe ratio test：

```
good_matches = [
    match for match, second in knn_matches
    if match.distance < ratio * second.distance
]
```

當 `ratio = 0.75` 時，必須滿足：

$$d_1 < 0.75d_2$$

其中：

- d_1 ：最佳匹配距離
- d_2 ：第二佳匹配距離

如果最佳匹配明顯比第二佳匹配更接近，才保留這個匹配。

ratio 參數的影響

```
ratio = 0.75
```

- ratio 較小，例如 `0.6`
 - 條件嚴格
 - 匹配較可靠
 - 但可能剩下太少匹配
- ratio 較大，例如 `0.85`
 - 條件寬鬆
 - 可以保留更多匹配
 - 但錯誤匹配也可能增加

4. Homography

匹配到特徵點之後，程式需要找出兩張影像之間的幾何轉換關係。

這個轉換稱為 Homography，可以表示為：

$$\begin{bmatrix} x' \\ y' \\ w' \end{bmatrix} = H \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

其中 H 是 3×3 的透視轉換矩陣。

程式使用：

```
homography, inlier_mask = cv2.findHomography(  
    source_points,  
    destination_points,  
    cv2.RANSAC,  
    5.0  
)
```

`source_points` 是第二張影像中的特徵點，`destination_points` 是第一張影像中對應的特徵點。

Homography 可以描述：

- 平移
- 旋轉
- 縮放
- 傾斜
- 透視變形

因此第二張影像可以被轉換到第一張影像的視角。

5. RANSAC 的作用

特徵匹配不可能全部正確。即使經過 ratio test，仍可能有錯誤匹配。

例如：

真正匹配：掛畫左上角 ↔ 掛畫左上角
錯誤匹配：牆面紋理 ↔ 窗戶邊緣

如果直接使用所有匹配點計算 Homography，錯誤匹配可能會讓整個轉換結果嚴重偏移。

RANSAC 的做法是反覆嘗試：

1. 隨機選取少量匹配點。
2. 計算一個 Homography。
3. 用這個 Homography 檢查其他匹配點。
4. 計算哪些點符合這個模型。
5. 選擇支持點最多的模型。

符合模型的點稱為 inliers，不符合的點稱為 outliers。

程式中的：

```
cv2.RANSAC,  
5.0
```

其中 `5.0` 是重投影誤差門檻。如果一個點經過 Homography 轉換後，和實際對應點的距離小於 5 個像素，就被視為內點。

RANSAC 內點比例

程式計算：

```
inlier_ratio = inlier_count / len(good_matches)
```

本次結果：

```
有效匹配數：139
RANSAC 內點數：128
內點比例：92.09%
```

這代表 139 個候選匹配中，有 128 個符合相同的幾何轉換，表示匹配品質很好。

內點比例越高，通常代表：

- 匹配越可靠
- Homography 越穩定
- 拼接結果越可信

6. Perspective Transform

取得 Homography 後，程式使用：

```
warped2 = cv2.warpPerspective(
    image2,
    transform,
    canvas_size
)
```

這會把第二張影像變形到第一張影像的座標系統。

因為轉換後影像可能出現負座標，所以程式先建立平移矩陣：

```
translation = np.array([
    [1, 0, -min_x],
    [0, 1, -min_y],
    [0, 0, 1],
], dtype=np.float64)
```

最後使用：

```
transform = translation @ homography
```

這樣可以確保轉換後的影像完整放在畫布範圍內。

7. 影像融合

第二張影像轉換後，程式先把第一張影像放進畫布：

```
result[first_y:first_y + height1,
       first_x:first_x + width1] = image1
```

在兩張影像重疊的區域，使用平均值融合：

```
result[overlap] = (
    0.5 * result[overlap].astype(np.float32)
    + 0.5 * warped2[overlap].astype(np.float32)
).astype(np.uint8)
```

這樣可以稍微減少兩張影像之間的亮度差異與接縫。

不過這是比較簡單的融合方式，可能仍會看到：

- 接縫
- 亮度不一致
- 物體重量
- 黑色背景區域

更進階的方法可以使用 feather blending 或 OpenCV 的 Stitcher。

8. CLAHE 前處理

進階實驗中使用 CLAHE：

```
clahe = cv2.createCLAHE(  
    clipLimit=2.0,  
    tileGridSize=(8, 8)  
)  
  
gray1 = clahe.apply(gray1)  
gray2 = clahe.apply(gray2)
```

CLAHE 是 Contrast Limited Adaptive Histogram Equalization，中文可稱為限制對比度自適應直方圖均衡化。

它和一般 Histogram Equalization 不同的地方是：

- 將影像分成許多小區塊
- 每個區塊個別調整對比
- 限制對比增強程度，避免雜訊過度放大
- 再將區塊結果平滑接合

CLAHE 特別適合：

- 光線不均勻
- 局部區域太暗
- 影像有陰影
- 全域對比不足

但 CLAHE 不一定總是改善結果。它也可能：

- 放大雜訊
- 產生太多錯誤特徵
- 讓匹配數增加但內點比例下降

因此需要搭配 RANSAC 內點比例一起判斷。

9. 進階實驗的意義

程式測試了幾種條件：

原始影像
亮度增加 60
亮度降低 60
旋轉 8 度
旋轉 20 度
減少重疊範圍
CLAHE 前處理

實驗結果：

條件	匹配數	內點數	內點比例	結果
原始影像	139	128	92.09%	成功
亮度增加 60	141	128	90.78%	成功
亮度降低 60	5	0	0%	失敗
旋轉 8 度	163	146	89.57%	成功
旋轉 20 度	143	125	87.41%	成功
減少重疊範圍	137	128	93.43%	成功
CLAHE	138	117	84.78%	成功

實驗結論

本次最明顯的失敗條件是「亮度降低 60」。

影像變暗後，SIFT 找到的穩定特徵大幅減少，因此有效匹配從 139 個降到 5 個，RANSAC 也無法找到可靠的 Homography。

旋轉 20 度仍然成功，代表 SIFT 對旋轉具有一定穩定性。減少重疊範圍也仍然成功，因為兩張照片仍保留足夠的共同特徵。

CLAHE 雖然可以改善局部對比，但本次內點比例反而從 92.09% 降到 84.78%。這說明前處理不是越強越好，應該實際比較：

- 有效匹配數
- RANSAC 內點數
- 內點比例
- 最終拼接影像品質